

Bawi Shopping — Deployment and Launch Guide

Bawi Shopping — Deployment and Launch Guide

This is the operational counterpart to `docs/IMPLEMENTATION-PLAN.md`. That document tracks what's been *built*; this one tracks what a human operator does to actually *run it* — every environment variable, every third-party service, every production setting, every feature flag, the deployment sequence, backups, monitoring, and the exact process for moving from mock/test values to real ones.

Nothing in this document is aspirational — every setting, route, and script named below exists in this repository today and was verified against the actual source during this session. Where something genuinely isn't built yet (a worker process, an object-storage adapter, a monitoring stack), that's stated plainly rather than glossed over — see §9.

1. What gets deployed

Four deployables today (a fifth, `apps/workers`, is designed for but not built — see §9):

Deployable	What it is	Default port
<code>apps/backend</code>	Medusa 2.x API server — the system of record. Everything else is a thin client over it.	<code>9000</code>
<code>apps/storefront</code>	Customer-facing Next.js app	<code>3000</code>
<code>apps/seller-portal</code>	Seller-facing Next.js app	<code>3001</code>
<code>apps/admin</code>	Bawi staff admin app, including the courier portal at <code>/courier/*</code>	<code>3002</code>

Each frontend is a standard Next.js app (`npm run build && npm run start`, or deploy to any Next.js-compatible host). The backend is a standard Node/Medusa process — deploy it anywhere that runs a long-lived Node process with outbound network access to Postgres and Stripe.

2. Every environment variable, by app

2.1 `apps/backend`

Variable	Required	Secret	Purpose
<code>DATABASE_URL</code>	Yes	Yes (contains credentials)	Postgres connection string. One dedicated database per environment (local/staging/production) — never shared.
<code>STORE_CORS</code>	Yes	No	Comma-separated allowed origins for the Store API (storefront's origin).

Variable	Required	Secret	Purpose
ADMIN_CORS	Yes	No	Comma-separated allowed origins for the Admin API (admin app's origin, plus Medusa's own admin dashboard origin if used).
AUTH_CORS	Yes	No	Comma-separated allowed origins for <code>/auth/*</code> routes — must include storefront, seller-portal, and admin origins.
JWT_SECRET	Yes	Yes	Signs session JWTs for every actor type (<code>customer</code> , <code>seller_user</code> , <code>user</code> , <code>courier</code>). Generate a long random string per environment; never reuse the placeholder in <code>.env.example</code> , never reuse across environments.
COOKIE_SECRET	Yes	Yes	Signs session cookies. Same rule as <code>JWT_SECRET</code> — unique per environment.
AUTH_MFA_ENCRYPTION_KEY	Yes	Yes	A 64-character hex string used by Medusa's auth module. Generate with <code>openssl rand -hex 32</code> .
SELLER_PORTAL_URL	Yes	No	Base URL the backend uses to build the seller account-activation link (<code>docs/MARKETPLACE-FLOWS.md</code>). Must be the real seller-portal origin in each environment.
COURIER_PORTAL_URL	Yes	No	Base URL for the courier portal, which lives inside <code>apps/admin</code> at <code>/courier</code> (not a separate app — see <code>docs/DECISIONS.md</code>). In production this is <code><admin-app-origin>/courier</code> .
ENABLE_TEST_SUPPORT_ROUTES	Yes	No	Must be <code>false</code> (or unset) in staging and production. <code>true</code> exposes <code>/seller-test-support/*</code> routes and switches Stripe to a network-free fake client — for local dev and CI only. Treat a <code>true</code> value reaching production as a security incident, not a config typo.
STRIPE_SECRET_KEY	Yes (once live)	Yes	Stripe platform account secret key. Test-mode (<code>sk_test_...</code>) until the go-live decision in §8; live (<code>sk_live_...</code>) only after that decision, and only ever set directly in the hosting provider's secret store — never committed, never in a shared <code>.env</code> file.
STRIPE_WEBHOOK_SECRET	Yes (once live)	Yes	Signing secret for the <code>/webhooks/stripe</code> endpoint (see §2.4). One value per registered webhook endpoint — staging and production have different endpoints and therefore different secrets.
REDIS_URL	Recommended for production	Yes (may contain credentials)	Not required for a single-instance deployment today (no code path depends on it yet — see §9). Required the moment you run more than one backend instance; see §9's event-bus/locking note.

2.2 apps/storefront

Variable	Required	Secret	Purpose
MEDUSA_BACKEND_URL	Yes	No	The backend's public URL.

Variable	Required	Secret	Purpose
<code>MEDUSA_PUBLISHABLE_KEY</code>	Yes	No (publishable by design)	Created in the Medusa admin (Settings → Publishable API Keys) and scoped to the storefront's sales channel. Regenerate per environment — don't reuse a staging key in production.
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	Yes (once checkout takes real cards)	No (publishable by design)	Test-mode (<code>pk_test_...</code>) or live (<code>pk_live_...</code>) Stripe publishable key, mounted client-side for the Payment Element. Never put a secret key in a <code>NEXT_PUBLIC_*</code> variable.

2.3 `apps/seller-portal` and `apps/admin`

Both need only:

Variable	Required	Secret	Purpose
<code>MEDUSA_BACKEND_URL</code>	Yes	No	The backend's public URL.

Neither app talks to Stripe directly — Stripe Connect onboarding and payment handling are entirely backend-mediated.

2.4 Values that aren't environment variables but must exist before launch

- **Stripe webhook endpoint**, registered in the Stripe Dashboard against the backend's `POST /webhooks/stripe`, subscribed at minimum to `payment_intent.succeeded`, `payment_intent.payment_failed`, `account.updated` (Connect), `charge.dispute.created`, and `charge.dispute.closed` — see `docs/PAYMENTS.md` for the full event list this handler processes. Staging and production each need their **own** endpoint (and therefore their own `STRIPE_WEBHOOK_SECRET`).
- A **Medusa admin user**, created via `npx medusa user -e you@example.com -p <password>` against the target environment's database — this is how the first admin logs into `apps/admin` and everything else (invites, seller staff, translations) branches from there.
- A **publishable API key + sales channel** in the Medusa admin, consumed by the storefront's `MEDUSA_PUBLISHABLE_KEY`.

3. Third-party services

Service	Status	Notes
PostgreSQL	Required, wired	One instance, three databases (<code>bawi_shopping_dev</code> / <code>_test</code> locally; one per real environment). Any managed Postgres 14+ works — local dev uses Postgres.app on port <code>5544</code> (<code>docs/DECISIONS.md</code>), which has no bearing on hosted environments.
Stripe (platform account + Connect Express)	Required, wired	One Stripe account, test mode until §8's go-live decision. See <code>docs/PAYMENTS.md</code> for the full commission/transfer/refund model.

Service	Status	Notes
Redis	Optional today, recommended before scaling past one backend instance	Not required for anything currently built — the backend runs today with Medusa's in-memory event bus and in-memory locking provider (confirmed by the literal startup warnings: <code>Local Event Bus installed. This is not recommended for production.</code> / <code>Locking module: Using "in-memory" as default.</code>). Safe for a single-instance deployment; not safe for more than one instance — see §9.
Object storage (S3-compatible)	Not wired yet	Product images currently go through Medusa's native <code>file</code> module with the local disk provider — fine for one instance with persistent disk, unsuitable for anything horizontally scaled or ephemeral (containers, serverless). See §9 for the exact config-only swap to <code>@medusajs/file-s3</code> .
Transactional email provider	Not wired yet	Emails today go through the native <code>notification</code> module's <code>notification-local</code> provider, which only logs — no email actually leaves the system, gated by <code>real_email_enabled</code> staying <code>false</code> . Swapping in a real provider (Medusa ships <code>@medusajs/notification-sendgrid</code> ; any SMTP/API provider works) is a business decision on top of an engineering one — see §8.
SMS provider	Not selected	<code>business-config</code> 's <code>sms.provider</code> is a placeholder (<code>mock_disabled</code>); <code>real_sms_enabled</code> stays <code>false</code> until one is chosen and wired.
Tax provider	Not selected	<code>tax.provider</code> is a placeholder (<code>mock</code>) using a flat 8.25% rate; <code>real_tax_calculation_enabled</code> stays <code>false</code> until a real jurisdiction-aware provider replaces the mock adapter in <code>src/tax/</code> .
Courier/delivery provider	Not selected	<code>courier.provider</code> is a placeholder (<code>mock_bawi_courier</code>); today couriers are admin-provisioned accounts using the app's own pickup/tracking-code flow, not an external logistics API. <code>real_courier_booking_enabled</code> stays <code>false</code> .
CDN	Not required, recommended	For serving product images once object storage is wired (§9) — not needed for the current local-disk setup.
Error tracking / APM / uptime monitoring	Not wired	See §9's monitoring section — nothing beyond structured stdout logging exists in the codebase today.

4. Production settings and feature flags

All business-configuration values live in one place: the `business-config` module (`apps/backend/src/modules/business-config/`), stored as `{category, key, value, is_placeholder}` rows, audit-logged on every change. **Never hardcode a business value in code** — add or edit a `business-config` entry instead.

4.1 How to view and change a value

- **Admin UI:** `apps/admin` → `/config` (`apps/admin/src/features/business-config/`) — lists every entry with its current value and `is_placeholder` status, and lets an authenticated admin user edit one.

- **API directly:** GET /admin/business-config (list all) and PUT /admin/business-config/:category/:key (update one), both requiring an admin user session.
- **Read-only report:** npx medusa exec ./src/scripts/check-production-readiness.ts (see §8) — prints every entry still at its placeholder default plus the state of every feature flag. This is a report, not a gate; nothing in CI or the deploy path currently blocks a deploy on its output (see §8's closing note on making that automatic).

4.2 Every business-config entry, its default, and whether it's a placeholder

Category	Key	Default	Placeholder?	What flipping it means
commission	platform_default_rate_basis_points	1500 (15.00%)	Yes	The commission rate applied to every sale absent a seller- or category-level override. A pricing/business decision.
transfer_timing	transfer_hold_days	7	Yes	Days after confirmed delivery before a seller's earnings become payable.
returns	return_window_days	14	Yes	Days after delivery a customer may request a return.
returns	return_shipping_policy	customer_pays_unless_defective	Yes	Who pays return shipping.
shipping	standard_shipping_fee_cents	699	Yes	Flat shipping fee. The one remaining placeholder as of this session — see §8.
shipping	free_shipping_threshold_cents	7500	Yes	Order subtotal above which shipping is free.
preparation	seller_preparation_deadline_hours	48	Yes	Hours a seller has to mark an order ready for pickup.
cancellation	cancellation_cutoff	preparing	Yes	Latest fulfillment stage at which a customer may still cancel.
service_area	initial_service_area	Dallas-Fort Worth, Texas	Yes	Launch geography.
brand_visibility	vendor_brand_requires_approval	true	No — v1 scope decision	Whether a seller's storefront brand name needs Bawi approval before going public.
payment_methods	currency	USD	No	Settlement currency.
payment_methods	accepted_methods	["stripe_test_cards", "stripe_test_wallets"]	No	Cosmetic/informational; real Stripe payment-method types are configured in the Stripe Dashboard, not here.

Category	Key	Default	Placeholder?	What flipping it means
tax	provider	mock	Yes	See §3.
tax	mock_rate_basis_points	825 (8.25%)	Yes	Flat placeholder tax rate applied uniformly regardless of address, until a real provider replaces the adapter.
courier	provider	mock_bawi_courier	Yes	See §3.
email	provider	mock_local	Yes	See §3.
sms	provider	mock_disabled	Yes	See §3.
support	support_email	support@example.bawishopping.com	Yes	Real support inbox needed before launch.
support	support_phone	+1-555-0100-TEST	Yes	A clearly fake placeholder — real support line needed before launch.
cart	cart_expiration_days	30	No — v1 scope decision	Days of inactivity before a cart is treated as abandoned.
cart	max_quantity_per_line_item	10	No — v1 scope decision	Per-line-item quantity cap.

4.3 Every feature flag

All nine default `false` and gate a code path that would otherwise move real money, send a real communication, or book a real courier — flipping any of them is a launch decision, not a config tweak done casually:

Flag	Gates
<code>live_payments_enabled</code>	Whether checkout is allowed to run against Stripe live mode at all — the master switch.
<code>real_transfers_enabled</code>	Whether commission-ledger transfers to sellers use real Stripe Connect transfers vs. staying test-mode.
<code>real_payouts_enabled</code>	Whether admin-triggered payout batches move real money.
<code>real_refunds_enabled</code>	Whether approved returns/cancellations issue real Stripe refunds.
<code>real_tax_calculation_enabled</code>	Whether checkout uses a real tax provider instead of the flat mock rate.
<code>real_email_enabled</code>	Whether notifications actually send email instead of only logging via <code>notification-local</code> .
<code>real_sms_enabled</code>	Whether SMS sends for real (no SMS provider is wired yet — see §3).
<code>real_courier_booking_enabled</code>	Whether fulfillment books a real external courier instead of using the app's own admin-assigned courier flow.

Flag	Gates
<code>promotional_codes_enabled</code>	Whether promotional/discount codes are accepted at checkout (deferred feature — see <code>docs/PRD.md §"Deferred Features"</code>).

5. Deployment steps

These steps apply to standing up any environment (staging or production) from scratch. Repeat per environment — never share a database, Stripe account, or set of secrets across environments.

1. **Provision Postgres.** Create a dedicated database. Note its connection string for `DATABASE_URL`.
2. **Generate secrets.** `JWT_SECRET`, `COOKIE_SECRET` (long random strings), `AUTH_MFA_ENCRYPTION_KEY` (`openssl rand -hex 32`) — unique to this environment, stored in the hosting provider's secret manager, never in a committed file.
3. **Set backend environment variables** (§2.1) — CORS origins pointing at this environment's actual frontend URLs, `ENABLE_TEST_SUPPORT_ROUTES` unset or `false`, Stripe keys in **test mode** initially even in a "production" environment (see §8 — going live with Stripe is a separate, deliberate step after the environment itself is verified working).
4. **Deploy the backend** and run `npx medusa db:migrate` against it — this applies every module's migrations in one pass and is also this project's own smoke test for workflow-definition errors (see `docs/DECISIONS.md`). Confirm it exits cleanly before proceeding.
5. **Create the first admin user:** `npx medusa user -e you@example.com -p <a-real-password>` against this environment.
6. **Log into `apps/admin`**, go to Settings → Publishable API Keys, create one scoped to the storefront's sales channel — this becomes the storefront's `MEDUSA_PUBLISHABLE_KEY`.
7. **Register the Stripe webhook endpoint** for this environment against `<backend-url>/webhooks/stripe` (see §2.4 for the event list); copy the resulting signing secret into `STRIPE_WEBHOOK_SECRET`.
8. **Deploy the three frontends** (`apps/storefront`, `apps/seller-portal`, `apps/admin`), each with its own `MEDUSA_BACKEND_URL` pointing at step 4's backend, and the storefront additionally with `MEDUSA_PUBLISHABLE_KEY` (step 6) and `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`.
9. **Update the backend's `SELLER_PORTAL_URL` and `COURIER_PORTAL_URL`** to this environment's real seller-portal and admin origins (courier portal lives at `<admin-url>/courier`), and redeploy the backend if these weren't already set correctly in step 3.
10. **Smoke test end to end:** register a customer, submit and approve a seller application, activate the seller, create and approve a product, add to cart, complete a test-mode checkout, confirm the Stripe webhook fires and splits the order, confirm a notification appears in-app. This exercises every module built across all eleven vertical slices in one pass.
11. **Run the production-readiness check** (§8) and review its output before considering this environment "launch-ready" for real money.

6. Backup strategy

- **Database:** Postgres is the sole system of record — no other durable store exists. Use your hosting provider's automated daily snapshots plus point-in-time recovery (WAL archiving) if available; this project doesn't ship its own backup tooling or schedule, since that's an infrastructure-provider responsibility, not application code.
- **Migrations, not manual DDL:** every schema change is a checked-in migration (`docs/DATABASE.md` §6) — restoring a backup and replaying migrations forward is always possible; there is no hand-edited schema drift to reconcile.
- **Object storage** (once wired per §9): back up the bucket itself via the provider's versioning/replication — product images aren't reproducible from the database alone.
- **Audit log:** the `audit-log` module records every approval/rejection/financial/role-change action and is itself just Postgres rows — covered by the database backup above, with no separate retention policy defined yet. If a compliance-driven retention/export requirement emerges, that's a business decision to make explicit in `business-config`, not an assumption to bake in silently.
- **Stripe** is its own system of record for payment/transfer/dispute history — Stripe's dashboard and API are the source of truth there, not a local mirror; this project's ledger tables are a derived view (see `docs/PAYMENTS.md`), not the canonical record.
- **No automated restore drill exists** — this is a real gap, not a built feature. Test a restore in staging before trusting it in production; that's a one-time operational exercise this document can name but not perform for you.

7. Monitoring setup

Nothing beyond structured stdout logging (Medusa's built-in logger, used throughout every module and workflow in this codebase) exists today. Before a real launch, at minimum:

- **Uptime / health checks** against the backend and each frontend — Medusa exposes a basic health endpoint; verify it responds and wire an external uptime monitor (any provider) to page on failure.
- **Error tracking** (e.g., an APM/error-tracking SDK) in the backend and all three Next.js apps — none is installed currently; this is a genuine gap worth closing before production traffic, not something silently assumed to exist.
- **Database monitoring** — connection count, query latency, disk usage — via whatever your Postgres host provides natively.
- **Stripe Dashboard** — webhook delivery success/failure, dispute rate, payout status — Stripe's own dashboard already covers this; no custom mirroring is needed unless you want it surfaced inside `apps/admin` too (not built, would be a new feature, not a gap in the current scope).
- **Log aggregation** — ship stdout from the backend and frontends to whatever log platform your host provides; nothing in this codebase writes logs anywhere other than stdout/stderr today.

8. Switching from mock/test values to real production values

Do this **only after** §5's environment is fully deployed and smoke-tested, and only with explicit authorization — this is a business/financial decision, not a code change, per this project's own non-negotiable rule that

`live_payments_enabled` and every `real_*` flag stay `false` until an explicit, separate production-launch approval (`CLAUDE.md` , `docs/DECISIONS.md`).

1. **Run the readiness report first:** `npm run medusa exec ./src/scripts/check-production-readiness.ts` . As of this session it reports exactly one remaining placeholder (`shipping.standard_shipping_fee_cents`) and every feature flag `false` . Do not proceed past this step while it reports any placeholder you haven't consciously reviewed and either replaced or explicitly accepted as a genuine v1 scope decision (see §4.2's "No — v1 scope decision" rows for the two that are fine to leave as-is).
2. **Replace every remaining `is_placeholder: true` business-config value** with a real, approved business decision via the admin `/config` UI or `PUT /admin/business-config/:category/:key` (§4.1) — commission rate, shipping fee, return window, support contact info, service area, and so on. Each edit is audit-logged automatically.
3. **Select and wire real provider integrations** for anything whose flag you intend to flip, one at a time, in this order (each is independent — you don't have to do all of them together):
 - **Email:** swap `@medusajs/notification-local` for a real provider (e.g. `@medusajs/notification-sendgrid` , or any SMTP/API provider) in `medusa-config.ts` 's `notification` module block, set `email.provider` in business-config, then flip `real_email_enabled` .
 - **Object storage** (needed before real product-image traffic at scale, independent of the flags above): add `@medusajs/file-s3` to the `modules` array in `medusa-config.ts` in place of the default local `file` module, and configure its S3 credentials/bucket — no application code change, config only (`docs/ARCHITECTURE.md` §6).
 - **Tax:** select and integrate a real jurisdiction-aware tax provider behind `src/tax/` 's existing adapter interface, set `tax.provider` , then flip `real_tax_calculation_enabled` .
 - **SMS:** select a provider, wire it, set `sms.provider` , then flip `real_sms_enabled` .
 - **Courier booking:** select a real logistics API if/when the in-app pickup/tracking-code model (already built and working) is no longer sufficient, set `courier.provider` , then flip `real_courier_booking_enabled` .
4. **Swap Stripe test keys for live keys** — set `STRIPE_SECRET_KEY` to a real `sk_live_...` value and register a **new**, separate live-mode webhook endpoint in the Stripe Dashboard (live and test mode webhooks are entirely separate registrations), updating `STRIPE_WEBHOOK_SECRET` accordingly. Do this directly in the hosting provider's secret store, never in a shared file.
5. **Flip `real_transfers_enabled` , `real_payouts_enabled` , and `real_refunds_enabled`** once you've confirmed a real Connect transfer/payout/refund behaves as expected against a real (but small, controlled) test transaction.
6. **Flip `live_payments_enabled` last** — this is the master switch that allows checkout to run against Stripe live mode at all. Treat this as the actual go-live moment.
7. **Re-run the readiness check** after every flag flip to confirm the change was applied where you intended and nothing else drifted.

8. **This process is entirely manual today** — nothing in CI or the deploy pipeline currently blocks a deploy on the readiness script's output (`docs/IMPLEMENTATION-PLAN.md` 's Phase 12 row notes this explicitly). Wiring the readiness check into CI as a hard gate before any environment's `live_payments_enabled` can be set is a reasonable follow-up, but is itself a process/policy change worth its own explicit decision rather than something to add silently.

9. Known gaps, stated plainly

These are genuine, current gaps — not oversights hidden from this document, and not things to "fix" unilaterally without confirming scope first:

- **No background worker process.** `apps/workers` is designed (`docs/ARCHITECTURE.md` §7) but not built. Payout batches, and every other action that might eventually be a scheduled job, are admin-triggered actions today. This is fine at v1 scale; revisit if/when payout volume or reconciliation needs true scheduling.
- **Single-instance assumption.** Running more than one backend instance without Redis-backed event bus (`@medusajs/event-bus-redis`), locking (`@medusajs/locking-redis`), and workflow engine (`@medusajs/workflow-engine-redis`) risks the exact correctness issues those modules exist to prevent — cross-instance workflow step duplication, race conditions on distributed locks. These packages are present in the dependency tree (part of the broader Medusa suite) but are **not currently added to `apps/backend/package.json` or configured in `medusa-config.ts`** — configuring them is a required step before scaling past one instance, not optional hardening.
- **Local-disk file storage.** Fine for one instance with a persistent volume; unsuitable for ephemeral/serverless/multi-instance hosting without first swapping to `@medusajs/file-s3` (§8, step 3).
- **No automated restore drill, no wired monitoring/error-tracking stack** (§6, §7) — both are real, named gaps for a human operator to close, not silently assumed handled.
- **The production-readiness check is a report, not a gate** — nothing currently stops a deploy or a flag flip if it reports a placeholder. See §8's closing note.